

Week1 实验报告

报告人：张筱涵

1. 实验思路

1.1 项目背景

Chooser 期权是一种特殊的金融衍生品，允许持有者在未来某一特定时间点（选择日）自主决定将其转换为看涨期权或看跌期权。这种灵活性使其在结构化产品设计、风险对冲及量化研究中具有重要价值。

传统定价方法依赖 Black-Scholes-Merton (BSM) 模型，其核心假设是波动率恒定。

$$C = S_0 N(d_1) - Xe^{-rT} N(d_2)$$

$$d_1 = \frac{\ln(S_0/X) + (r + \frac{\sigma^2}{2})T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

然而现实市场中波动率随市场状态持续变化，BSM 模型的定价误差会显著放大。因此本项目旨在通过引入真实市场数据与机器学习方法，构建一个能够动态捕捉波动率变化的增强定价模型。

1.2 数据需求分析

为了解决 BSM 模型的局限性，本项目将特征空间从 BSM 的 5 个基本参数扩展至多维度数据体系，数据来源分为以下三类：

标的资产数据 (JPM 股价)

选择 JPMorgan Chase (JPM) 作为标的资产，原因有三：

- JPM 是高流动性的大盘金融股，期权市场活跃，数据质量高；
- 金融股对美联储利率决策高度敏感，在 FOMC 决议前后波动率特征显著，适合测试制度依赖型定价模型；
- JPM 是 BSM 相关学术文献中的常用标的，便于与已有研究结果进行对比验证。

波动率数据 (VIX 指数)

VIX 是芝加哥期权交易所发布的市场隐含波动率指数，代表市场对未来 30 天波动率的共识预期。VIX 既可作为 BSM 中 σ 参数的市场代理，也是机器学习模型识别市场恐慌制度的核心特征。通过引入 VIX 数据，模型能够感知当前市场处于低波动、中等波动还是高波动状态，从而动态调整定价参数。

美国国债收益率

BSM 模型需要无风险利率 r 作为输入参数。本项目收集了 3 个月至 10 年期的完整国债收益率期限结构，而非单一固定利率，因为不同到期日的期权需要匹配对应期限的无风险利率，完整的收益率曲线可以通过插值为任意到期日提供精确的 r 值。此外，收益率曲线的形态（如 10 年期与 2 年期利差）是判断经济衰退风险的经典指标，对波动率的系统性变化具有预测价值。

2. 整体流程设计及关键代码

本周按照以下四个阶段进行：

2.1 API 连接与数据收集

配置并测试三个数据源的 API 连接，包括 Yahoo Finance（JPM 股价、VIX）、FRED（国债收益率及宏观经济指标）和 Alpha Vantage（备用数据源）。确认连接正常后，统一收集 2018 年至 2024 年的日频历史数据。

```
START = "2018-01-01"
END = "2024-12-31"
fred = Fred(api_key="793ea442d5187f9e5b7b0c30de96367a")
# 1. JPM
print("JPM data...")
jpm = yf.download("JPM", start=START, end=END)
jpm.columns = [c[0] for c in jpm.columns]
print(f"JPM: {len(jpm)} row")
# 2. VIX
print("VIX data...")
vix = yf.download("^VIX", start=START, end=END, auto_adjust=True)
vix.columns = [c[0] for c in vix.columns]
print(f"VIX: {len(vix)} row")
# 3. Treasury yield
print("Treasury data...")
treasury_series = {
    "DGS3M0": "Yield_3M",
    "DGS1": "Yield_1Y",
    "DGS2": "Yield_2Y",
    "DGS10": "Yield_10Y"
}
treasury = pd.DataFrame()
for code, name in treasury_series.items():
    s = fred.get_series(code, observation_start=START,
observation_end=END)
    treasury[name] = s
print(f"Treasury yield: {len(treasury)} row ")
```

运行结果：

```
JPM data...
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
JPM: 1760
VIX data...
VIX: 1760
Treasury data...
Treasury yield: 1827
```

2.2 数据清洗与时间对齐

由于三类数据来源不同，存在频率不一致（国债数据含非交易日）和缺失值等问题。以 JPM 交易日为基准建立统一时间轴，对国债数据采用前向填充处理频率差异，对缺失值采用插值法处理，并通过 IQR 方法识别和处理异常值，最终合并为 1760 行×6 列的主数据集。

```
# JPM 只保留收盘价
jpm_close = jpm[["Close"]].copy()
jpm_close.columns = ["JPM_Close"]
jpm_close.index = pd.to_datetime(jpm_close.index)
# VIX 只保留收盘价
vix_close = vix[["Close"]].copy()
vix_close.columns = ["VIX_Close"]
vix_close.index = pd.to_datetime(vix_close.index)
# 国债统一索引格式
treasury.index = pd.to_datetime(treasury.index)
# 合并，以 JPM 交易日为基准
master = jpm_close.copy()
master = master.join(vix_close, how="left")
master = master.join(treasury, how="left")
# 填充国债缺失值
master = master.ffill()
master = master.dropna()
print(f"主数据集: {master.shape[0]} row x {master.shape[1]} column")
print(master.tail())
```

最近日期的运行结果：

```
主数据集: 1760 row x 6 column
      JPM_Close  VIX_Close  Yield_3M  Yield_1Y  Yield_2Y  Yield_10Y
Date
2024-12-23  231.216553  16.780001      4.36      4.26      4.30      4.59
2024-12-24  235.018570  14.270000      4.40      4.24      4.29      4.59
2024-12-26  235.823608  14.730000      4.35      4.23      4.30      4.58
2024-12-27  233.912872  15.950000      4.31      4.20      4.31      4.62
2024-12-30  232.118561  17.400000      4.37      4.17      4.24      4.55
```

2.3 特征工程

在主数据集基础上构建两类特征。传统特征直接对应 BSM 参数，包括日对数收益率、10/30/60 日滚动实现波动率，用于基准模型的参数估计。高级特征面向机器学习模型，包括 VIX 制度分类（Low/Medium/High）、利率动量、收益率曲线斜率（10Y-2Y 利差）及 VIX-JPM 滚动相关性，用

于捕捉 BSM 的非线性市场动态。最终生成 1697 行×14 列的特征数据集。

```
# 特征工程
df = master.copy()
# 1. 日收益率
df["Log_Return"] = np.log(df["JPM_Close"] / df["JPM_Close"].shift(1))
# 2. 滚动波动率 (annual)
df["RealVol_10d"] = df["Log_Return"].rolling(10).std() * np.sqrt(252)
df["RealVol_30d"] = df["Log_Return"].rolling(21).std() * np.sqrt(252)
df["RealVol_60d"] = df["Log_Return"].rolling(63).std() * np.sqrt(252)
# 3. VIX 制度分类
lo = df["VIX_Close"].quantile(0.33)
hi = df["VIX_Close"].quantile(0.66)
df["VIX_Regime"] = pd.cut(df["VIX_Close"],
                           bins=[-np.inf, lo, hi, np.inf],
                           labels=["Low", "Medium", "High"])
# 4. 利率动量 (10d)
df["Rate_Momentum"] = df["Yield_3M"].diff(10)
# 5. 收益率曲线斜率
df["Yield_Spread"] = df["Yield_10Y"] - df["Yield_2Y"]
# 6. VIX-JPM 滚动相关性
df["VIX_JPM_Corr"] = df["VIX_Close"].rolling(30).corr(df["JPM_Close"])
df = df.dropna()
df.to_csv("featured_dataset.csv")
```

最近日期的运行结果：

特征工程完成：1697 行 × 14 列
所有特征：['JPM_Close', 'VIX_Close', 'Yield_3M', 'Yield_1Y', 'Yield_2Y', 'Yield_10Y', 'Log_Return', 'RealVol_10d', 'RealVol_30d', 'RealVol_60d', 'VIX_Regime', 'Rate_Momentum', 'Yield_Spread', 'VIX_JPM_Corr']

	JPM_Close	VIX_Close	Yield_3M	Yield_1Y	Yield_2Y	Yield_10Y \
Date						
2024-12-23	231.216553	16.780001	4.36	4.26	4.30	4.59
2024-12-24	235.018570	14.270000	4.40	4.24	4.29	4.59
2024-12-26	235.823608	14.730000	4.35	4.23	4.30	4.58
2024-12-27	233.912872	15.950000	4.31	4.20	4.31	4.62
2024-12-30	232.118561	17.400000	4.37	4.17	4.24	4.55

	Log_Return	RealVol_10d	RealVol_30d	RealVol_60d	VIX_Regime \
Date					
2024-12-23	0.003319	0.224481	0.184694	0.300058	Medium
2024-12-24	0.016310	0.242516	0.185789	0.300885	Low
2024-12-26	0.003420	0.242767	0.184217	0.300764	Low
2024-12-27	-0.008135	0.242664	0.185694	0.301498	Medium
2024-12-30	-0.007700	0.243541	0.186798	0.302154	Medium

	Rate_Momentum	Yield_Spread	VIX_JPM_Corr
Date			
2024-12-23	-0.06	0.29	-0.717010
2024-12-24	-0.01	0.30	-0.725080
2024-12-26	-0.03	0.28	-0.738199
2024-12-27	-0.04	0.31	-0.749411
2024-12-30	0.03	0.31	-0.759023

2.3 自动化流水线搭建

在 GitHub 仓库中配置 Actions 工作流，设置每天凌晨 1 点自动触发数据收集脚本，将最新市场数据更新至仓库，实现数据管道的全自动化运行，为后续模型的实时定价功能奠定基础。

```
name: Auto Data Update

on:
  schedule:
    - cron: "0 1 * * *"
  workflow_dispatch:

permissions:
  contents: write

jobs:
  update-data:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
      - name: Install dependencies
        run: pip install yfinance pandas numpy fredapi
      - name: Run data collector
        run: python scripts/data_collector.py
      - name: Commit updated data
        run: |
          git config --global user.email "action@github.com"
          git config --global user.name "GitHub Action"
          git add data/
          git commit -m "Auto update $(date)" || echo "No changes"
          git push
```

自动化结果：调试完成，可以自行收集数据

← Auto Data Update

🟢 Auto Data Update #5

🏠 Summary

All jobs

🟢 update-data

Run details

🕒 Usage

📄 Workflow file

Manually triggered 4 minutes ago

👤 xzha0487 → 5f5b00e main

Status

Success

Total duration

30s

Artifacts

—

data_update.yml

on: workflow_dispatch

🟢 update-data

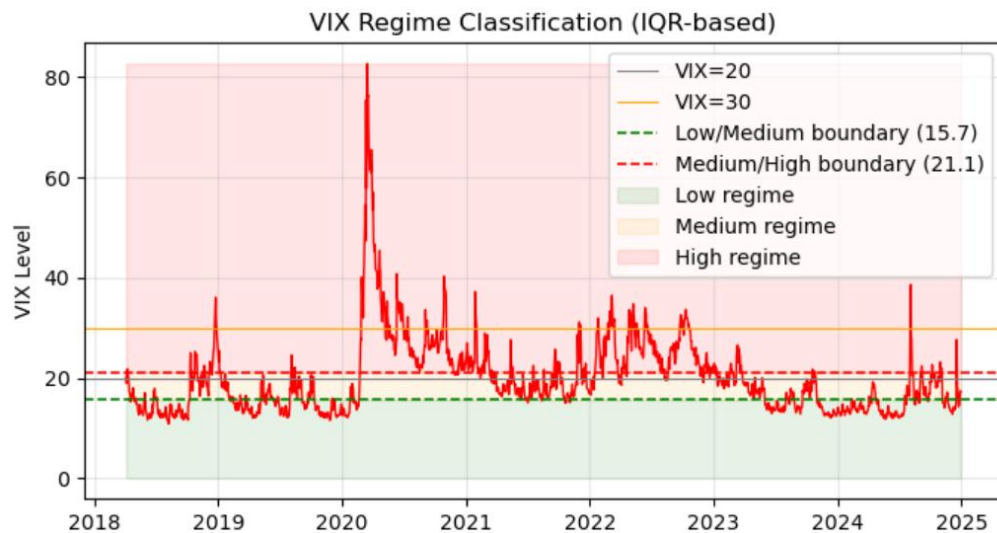
25s

3. 可视化图表

3.1 JPM 股价走势（2018-2024）

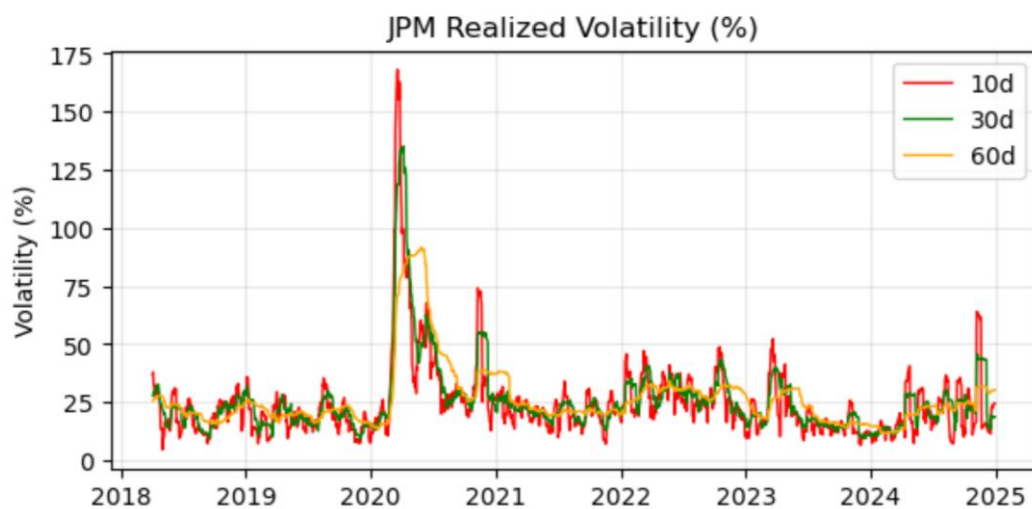


3.2 VIX 波动率指数走势

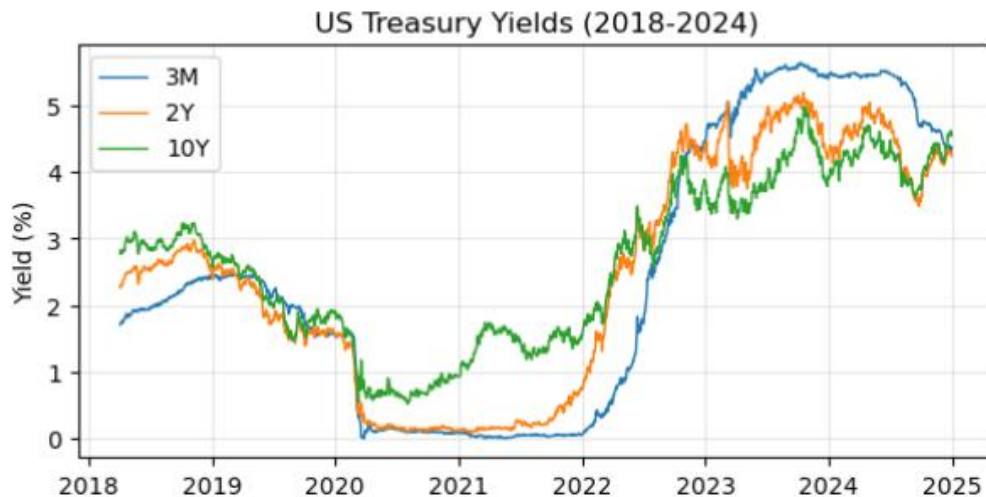


本图展示了 2018 年至 2024 年 VIX 波动率指数的走势及基于 IQR 三分位数的制度分类结果。根据数据分布，Low/Medium 分界值为 15.7，Medium/High 分界值为 21.1，将样本期划分为三个波动率制度区间。

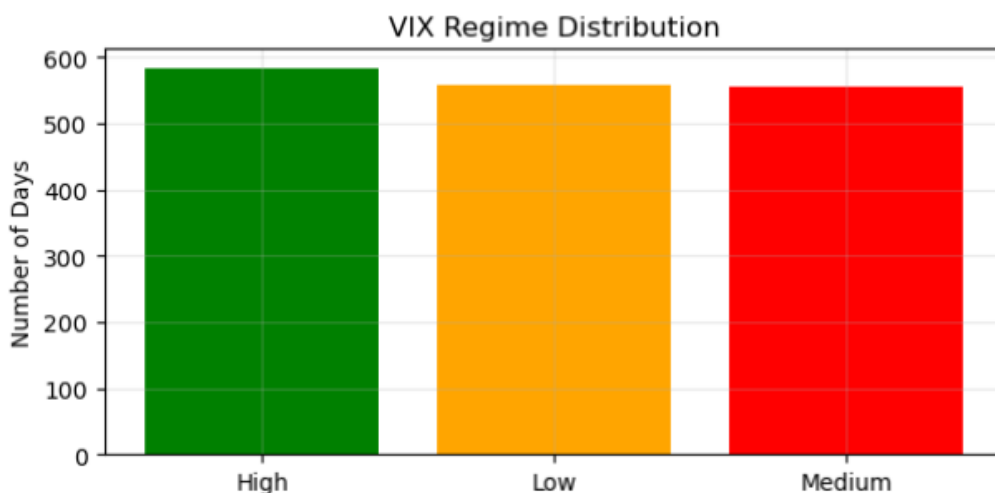
3.3 滚动波动率对比



3.4 美国国债收益率曲线



3.5 VIX 制度分类分布



4.下周任务计划

下周的核心任务是基于本周构建的数据集，复现论文中的 BSM Chooser 期权定价模型。首先按照论文参数实现 BSM 定价公式，设置行权价为 150 美元，期权到期时间 T 为 1 年，选择日为到期前 6 个月，在 Jupyter Notebook 中完整实现定价逻辑。随后将模型的模拟输出结果与论文中报告的数值进行对比验证，确认模型复现的准确性。

在 BSM 模型复现的基础上，计划通过 MAE、RMSE 等误差指标量化模型预测值与 CME 真实市场成交价格之间的偏差，重点分析模型在高波动率时期（如 2020 年）的失效情况，识别 BSM 恒定波动率假设在极端市场环境下的局限性。最终整理形成完整的基准性能报告，为后续机器学习模型的优化提供对比基准。